Phase 8 Structured Notes: Double Ended Queue

DEQueue / Deque

---

1. Current Progress Before Phase 8

Before Phase 8, you already learned the main queue ideas.

In **Phase 1**, you learned that a queue follows this rule:

```
FIFO = First In, First Out
```

That means:

```
The first value inserted into the queue is the first value removed from the
queue.
```

You also learned that a normal queue has two important ends:

Queue Part	Simple Meaning
`front`	The side where deletion happens
`rear`	The side where insertion happens

So in a normal queue:

```
Insert at rear.
Delete from front.
```

In **Phase 2**, you compared two array queue designs:

Design	Main Idea	Result
Single-pointer queue	Uses only `rear`	Deletion is slow because elements must shift
Two-pointer queue	Uses `front` and `rear`	Deletion is fast because only `front` moves

The important lesson was:

```
The two-pointer queue avoids shifting.
```

In **Phase 3**, you built a complete linear array queue in C using:

- `struct Queue`
- `size`
- `front`
- `rear`
- `Q`
- `create()`
- `enqueue()`
- `dequeue()`
- `display()`
- `malloc()`
- `free()`

In **Phase 4**, you implemented the same queue idea using C++ OOP.

You used:

- `class Queue`
- private data
- public methods
- constructors
- destructor
- `new[]`
- `delete[]`
- object syntax like `q.enqueue(10)`

In **Phase 5**, you learned the weakness of a linear array queue.

The problem was not speed.

The problem was:

```
Memory reuse.
```

A linear queue wastes space because `rear` only moves forward. Deleted spaces at the beginning of the array cannot be reused unless the queue becomes completely empty and resets.

In **Phase 6**, you solved this problem using a circular queue.

A circular queue allows `rear` and `front` to wrap around using modulo:

```
(rear + 1) % size
(front + 1) % size
```

In **Phase 7**, you learned a linked-list queue.

A linked-list queue stores data inside connected nodes instead of a fixed array.

It uses:

- `front` pointer to the first node
- `rear` pointer to the last node
- dynamic memory node by node

Now Phase 8 introduces a more flexible queue:

```
Double Ended Queue
```

It is also called:

```
DEQueue
Deque
```

---

2. Main Goal of Phase 8

The goal of Phase 8 is to understand a queue that works from **both ends**.

A normal queue is strict:

```
Insert only at rear.
Delete only from front.
```

A double ended queue is more flexible:

```
Insert at rear.
Delete from front.
Insert at front.
Delete from rear.
```

So the main idea of Phase 8 is:

```
Deque = queue with both ends active.
```

---

3. What Is a Double Ended Queue?

A **double ended queue** is a queue where insertion and deletion can happen from both sides.

It is called **double ended** because both ends can be used:

- the front end
- the rear end

A normal queue allows only two main actions:

```
insert at rear
remove from front
```

A deque allows four main actions:

```
insert at rear
remove from front
insert at front
remove from rear
```

---

4. Pronunciation of Deque

The word **deque** is usually pronounced like:

```
deck
```

Even though it is written as `deque`, it is commonly said like **deck**.

---

5. Normal Queue vs Deque

A normal queue works like this:

```
front [10, 20, 30] rear
```

In a normal queue:

```
enqueue(40)  -> insert at rear
dequeue()    -> delete from front
```

So a normal queue can be shown like this:

```
Deletion happens here        Insertion happens here
        ↓                              ↓
      front       [10, 20, 30]       rear
```

A deque is different.

In a deque, both sides can be used:

```
Insertion or deletion can happen here     Insertion or deletion can happen here
               ↓                                        ↓
             front          [10, 20, 30]            rear
```

Simple comparison:

Structure	Front Can Do	Rear Can Do
Normal queue	Delete only	Insert only
Deque	Insert and delete	Insert and delete

---

6. Important Difference Between Queue and Deque

A normal queue always follows FIFO:

```
First In, First Out
```

Example:

```
Inserted order: 10, 20, 30
Deleted order: 10, 20, 30
```

The first value inserted is `10`, so `10` must leave first.

But a deque does not always have to behave like a strict FIFO queue.

Why?

Because a deque can delete from the rear.

Example:

```
Deque: 10 20 30
```

If we call:

```
deleteRear()
```

Then `30` is removed first.

That means the newest value can leave first.

So remember:

```
A normal queue is always FIFO.
A deque can behave like a queue, but it can also delete from the rear.
```

---

7. The Four Main Deque Operations

A deque has four important operations:

Operation	Meaning
`insertRear(x)`	Insert value `x` at the rear
`deleteFront()`	Delete value from the front
`insertFront(x)`	Insert value `x` at the front
`deleteRear()`	Delete value from the rear

The first two operations are already familiar from normal queues:

```
insertRear()
deleteFront()
```

They behave like normal queue operations.

The new operations are:

```
insertFront()
deleteRear()
```

These new operations make the deque more flexible.

---

8. Simple Deque Example

Suppose the deque starts empty.

We insert values at the rear:

```
insertRear(10)
insertRear(20)
insertRear(30)
```

Now the deque is:

```
10 20 30
```

If we call:

```
deleteFront()
```

The deleted value is:

```
10
```

Now the deque is:

```
20 30
```

If we call:

```
insertFront(5)
```

Now the deque becomes:

```
5 20 30
```

If we call:

```
deleteRear()
```

The deleted value is:

```
30
```

Now the deque is:

```
5 20
```

This example shows all four main operations.

---

9. Array Implementation of Deque

In this phase, we can understand a deque using an array.

We use a structure like this:

```c
struct Deque {
    int size;
    int front;
    int rear;
    int *Q;
};
```

This is similar to the earlier array queue structure.

---

10. Meaning of Each Variable

Variable	Simple Meaning
`size`	Maximum number of values the deque can store
`front`	Position before the first valid element
`rear`	Position of the last valid element
`Q`	Pointer to the array that stores the values

Important point:

```
front and rear are index values in this array version.
They are not actual memory addresses.
```

Example:

```
front = 0
rear = 2
```

This means `front` and `rear` are positions inside the array.

---

11. Starting State of the Deque

At the beginning, the deque is empty.

We write:

```
front = -1;
rear = -1;
```

This means:

```
front = -1
rear = -1
```

Since both are equal, the deque is empty.

So the empty condition is:

```
front == rear
```

---

12. Important Mental Model for `front`

In this simple array design, `front` does not directly point to the first valid element.

Instead:

```
front points before the first valid element.
```

So the first valid element is at:

```
front + 1
```

Example:

```
front = -1
rear = 2
Array = [10, 20, 30]
```

The first valid element is at:

```
front + 1 = 0
```

So the first valid value is:

```
Q[0] = 10
```

The valid deque range is:

```
front + 1 to rear
```

---

13. Empty Condition

The deque is empty when:

```
front == rear
```

Example 1:

```
front = -1
rear = -1
```

The deque is empty.

Example 2:

```
front = 2
rear = 2
```

The deque is also empty because there are no valid elements between `front + 1` and `rear`.

---

14. Full Conditions in This Simple Linear Deque

Because this is a simple linear array version, the front and rear spaces are checked separately.

No Space at Rear

There is no space at the rear when:

```
rear == size - 1
```

This means `rear` is already at the last index.

Example:

```
size = 5
last index = 4
rear = 4
```

There is no more space at the rear.

No Space at Front

There is no space at the front when:

```
front == -1
```

This means there is no available index before the first valid element.

In this simple version, `insertFront()` usually becomes possible after something has been deleted from the front.

---

15. The `create()` Function

The `create()` function prepares the deque.

```
void create(Deque *dq, int size) {
    dq->size = size;
    dq->front = dq->rear = -1;
    dq->Q = new int[size];
}
```

Job of `create()`

The `create()` function does three main things:

1. Stores the deque size.
2. Sets `front` and `rear` to `-1`.
3. Creates the array in heap memory using `new[]`.

After calling:

```
create(&dq, 5);
```

The deque state is:

```
size = 5
front = -1
rear = -1
Q = array space for 5 integers
```

The deque is ready to use.

16. Why Do We Pass `&dq`?

In functions like `create()`, `insertRear()`, `deleteFront()`, `insertFront()`, and `deleteRear()`, we need to change the original deque.

That is why we pass the address:

```
create(&dq, 5);
insertRear(&dq, 10);
deleteFront(&dq);
```

The symbol `&` means:

```
address of
```

So `&dq` means:

```
the address of dq
```

This allows the function to modify the real deque, not a copy.

---

17. Why Do We Use `Deque *dq`?

A function like this:

```
void insertRear(Deque *dq, int x)
```

means:

```
dq is a pointer to a Deque structure.
```

Because `dq` is a pointer, we use the arrow operator:

```
dq->rear
```

This means:

```
Go to the deque that dq points to, then access rear.
```

Part A: `insertRear()`

---

18. What Is `insertRear()`?

`insertRear()` means:

```
Insert a value at the rear end of the deque.
```

This is the same as normal queue insertion.

Example:

```
Deque before: 10 20
Operation: insertRear(30)
Deque after: 10 20 30
```

The new value `30` is added at the rear.

---

19. `insertRear()` Code

```
void insertRear(Deque *dq, int x) {
    if (dq->rear == dq->size - 1) {
        cout << "No space at rear\n";
    } else {
        dq->rear++;
        dq->Q[dq->rear] = x;
    }
}
```

---

20. `insertRear()` Logic Step by Step

To insert at the rear:

1. Check whether there is space at the rear.
2. If `rear == size - 1`, there is no rear space.
3. Otherwise, move `rear` one step forward.
4. Store the new value at `Q[rear]`.

Simple meaning:

```
If rear is already at the last index, insertion at rear is not possible.
Otherwise, move rear forward and store the value.
```

---

21. Line-by-Line Explanation of `insertRear()`

Full check

```
if (dq->rear == dq->size - 1)
```

This checks whether `rear` is already at the last valid index.

Message

```
cout << "No space at rear\n";
```

This prints a message if insertion at the rear is not possible.

Move rear

```
dq->rear++;
```

This moves `rear` one step forward.

Store value

```
dq->Q[dq->rear] = x;
```

This stores the new value at the rear position.

22. `insertRear()` Dry Run

Start:

```
size = 5
front = -1
rear = -1
Array = [ ] [ ] [ ] [ ] [ ]
```

Call:

```
insertRear(&dq, 10);
```

Step 1: Check rear space.

```
rear == size - 1?
-1 == 4? No
```

Step 2: Move `rear`.

```
rear moves from -1 to 0
```

Step 3: Store value.

```
Q[0] = 10
```

Now:

```
front = -1
rear = 0
Deque = 10
```

Call:

```
insertRear(&dq, 20);
```

Now:

```
rear moves from 0 to 1
Q[1] = 20
Deque = 10 20
```

Call:

```
insertRear(&dq, 30);
```

Now:

```
rear moves from 1 to 2
Q[2] = 30
Deque = 10 20 30
```

---

23. Time Complexity of `insertRear()`

`insertRear()` is:

```
O(1)
```

Why?

Because it only:

- checks the rear position
- moves `rear`
- stores one value

It does not use a loop.

Part B: `deleteFront()`

---

24. What Is `deleteFront()`?

`deleteFront()` means:

```
Delete and return the value from the front end of the deque.
```

This is the same as normal queue deletion.

Example:

```
Deque before: 10 20 30
Operation: deleteFront()
Deleted value: 10
Deque after: 20 30
```

The value `10` is deleted because it is at the front.

---

25. `deleteFront()` Code

```
int deleteFront(Deque *dq) {
    if (dq->front == dq->rear) {
        cout << "Deque empty\n";
        return -1;
    }

    dq->front++;
    return dq->Q[dq->front];
}
```

---

26. `deleteFront()` Logic Step by Step

To delete from the front:

1. Check whether the deque is empty.
2. If `front == rear`, there is nothing to delete.
3. Otherwise, move `front` one step forward.
4. Return the value at `Q[front]`.

Simple meaning:

```
If the deque is empty, deletion is not possible.
Otherwise, front moves to the first valid element and returns it.
```

---

27. Line-by-Line Explanation of `deleteFront()`

Empty check

```
if (dq->front == dq->rear)
```

This checks whether the deque is empty.

Empty message

```
cout << "Deque empty\n";
```

This prints a message if there is nothing to delete.

Return failure value

```
return -1;
```

This returns `-1` if deletion is not possible.

Move front

```
dq->front++;
```

This moves `front` to the first valid element.

Return deleted value

```
return dq->Q[dq->front];
```

This returns the value being removed.

---

28. `deleteFront()` Dry Run

Suppose the deque is:

```
front = -1
rear = 2
```

```
Array = [10] [20] [30] [ ] [ ]
Logical deque = 10 20 30
```

Call:

```
deleteFront(&dq);
```

Step 1: Check empty.

```
front == rear?
-1 == 2? No
```

Step 2: Move `front`.

```
front moves from -1 to 0
```

Step 3: Return value.

```
Q[0] = 10
```

Deleted value:

```
10
```

Now:

```
front = 0
rear = 2
Logical deque = 20 30
```

The value `10` may still physically exist in the array, but it is no longer part of the logical deque.

---

29. Logical Deletion in Deque

Just like earlier array queues, deletion in this simple deque is logical.

Logical deletion means:

```
The old value may still remain in the array, but the deque ignores it.
```

Example before deletion:

```
front = -1
rear = 2
Array = [10, 20, 30]
```

After `deleteFront()`:

```
front = 0
rear = 2
Array may still be [10, 20, 30]
```

But the logical deque is:

```
20 30
```

Why?

Because the valid range is:

```
front + 1 to rear
```

That means:

```
1 to 2
```

So the valid values are:

```
Q[1] = 20
Q[2] = 30
```

The old value `10` is ignored.

---

30. Time Complexity of `deleteFront()`

`deleteFront()` is:

```
O(1)
```

Why?

Because it only:

- checks if the deque is empty
- moves `front`
- returns one value

It does not shift elements.

Part C: `insertFront()`

---

31. What Is `insertFront()`?

`insertFront()` means:

```
Insert a value at the front end of the deque.
```

This is one of the new operations in a deque.

Example:

```
Deque before: 20 30
Operation: insertFront(5)
Deque after: 5 20 30
```

The new value `5` is added before the old front value.

---

32. Why `insertFront()` Is Tricky

In this simple array design, `front` points before the first valid element.

So if:

```
front = -1
```

there is no available index before the first element.

That means insertion at the front is not possible.

But after `deleteFront()`, `front` moves forward.

Example:

```
Before deleteFront:
front = -1
rear = 2
Deque = 10 20 30
```

After deleting `10`:

```
front = 0
rear = 2
Deque = 20 30
```

Now index `0` is free logically.

So we can insert a new value at the front.

---

33. `insertFront()` Code

```
void insertFront(Deque *dq, int x) {
    if (dq->front == -1) {
        cout << "No space at front\n";
    } else {
        dq->Q[dq->front] = x;
        dq->front--;
    }
}
```

---

34. `insertFront()` Logic Step by Step

To insert at the front:

1. Check whether there is space at the front.
2. If `front == -1`, there is no front space.
3. Otherwise, store the new value at `Q[front]`.
4. Move `front` backward by one step.

Simple meaning:

```
Use the free space just before the first valid element.
Then move front back so it again points before the new first value.
```

---

35. Line-by-Line Explanation of `insertFront()`

Front space check

```
if (dq->front == -1)
```

This checks whether there is space before the first valid element.

If `front == -1`, there is no position before it.

Message

```
cout << "No space at front\n";
```

This prints a message if insertion at the front is not possible.

Store value

```
dq->Q[dq->front] = x;
```

This stores the new value at the current `front` index.

Move front backward

```
dq->front--;
```

This moves `front` back so it again points before the first valid element.

---

36. `insertFront()` Dry Run

Suppose we first insert values at the rear:

```
insertRear(&dq, 10);
insertRear(&dq, 20);
insertRear(&dq, 30);
```

Now:

```
front = -1
rear = 2
Array = [10] [20] [30] [ ] [ ]
Logical deque = 10 20 30
```

Now call:

```
deleteFront(&dq);
```

This deletes `10`.

Now:

```
front = 0
rear = 2
Logical deque = 20 30
```

Index `0` is now free logically.

Now call:

```
insertFront(&dq, 5);
```

Step 1: Check front space.

```
front == -1?
0 == -1? No
```

So there is space.

Step 2: Store value.

```
Q[front] = 5
Q[0] = 5
```

Step 3: Move `front` backward.

```
front moves from 0 to -1
```

Now:

```
front = -1
rear = 2
Array = [5] [20] [30] [ ] [ ]
Logical deque = 5 20 30
```

---

37. Important Beginner Point About `insertFront()`

In this simple linear array version, `insertFront()` cannot usually be used at the very beginning.

At the beginning:

```
front = -1
rear = -1
```

If we call:

```
insertFront(&dq, 10);
```

The function checks:

```
front == -1
```

This is true.

So it prints:

```
No space at front
```

This happens because there is no index before `-1`.

In this simple version, `insertFront()` becomes useful after `deleteFront()` has created space at the front.

---

38. Time Complexity of `insertFront()`

`insertFront()` is:

```
O(1)
```

Why?

Because it only:

- checks the front position
- stores one value
- moves `front` backward

It does not use a loop.

Part D: `deleteRear()`

---

39. What Is `deleteRear()`?

`deleteRear()` means:

```
Delete and return the value from the rear end of the deque.
```

This is also one of the new operations in a deque.

Example:

```
Deque before: 5 20 30
Operation: deleteRear()
Deleted value: 30
Deque after: 5 20
```

The value `30` is removed because it is at the rear.

---

40. `deleteRear()` Code

```
int deleteRear(Deque *dq) {
    if (dq->front == dq->rear) {
        cout << "Deque empty\n";
        return -1;
    }

    int x = dq->Q[dq->rear];
    dq->rear--;
    return x;
}
```

---

41. `deleteRear()` Logic Step by Step

To delete from the rear:

1. Check whether the deque is empty.
2. If `front == rear`, there is nothing to delete.
3. Otherwise, store the value at `Q[rear]` in `x`.
4. Move `rear` one step backward.
5. Return `x`.

Simple meaning:

```
Take the newest value from the rear, then move rear backward.
```

---

42. Line-by-Line Explanation of `deleteRear()`

Empty check

```
if (dq->front == dq->rear)
```

This checks whether the deque is empty.

Empty message

```
cout << "Deque empty\n";
```

This prints a message if there is nothing to delete.

Return failure value

```
return -1;
```

This returns `-1` if deletion is not possible.

Store rear value

```
int x = dq->Q[dq->rear];
```

This saves the rear value before moving `rear`.

Move rear backward

```
dq->rear--;
```

This removes the rear value logically.

Return deleted value

```
return x;
```

This returns the value that was removed.

---

43. `deleteRear()` Dry Run

Suppose the deque is:

```
front = -1
rear = 2
Array = [5] [20] [30] [ ] [ ]
Logical deque = 5 20 30
```

Call:

```
deleteRear(&dq);
```

Step 1: Check empty.

```
front == rear?
-1 == 2? No
```

Step 2: Store rear value.

```
x = Q[rear]
x = Q[2]
x = 30
```

Step 3: Move `rear` backward.

```
rear moves from 2 to 1
```

Deleted value:

```
30
```

Now:

```
front = -1
rear = 1
Logical deque = 5 20
```

---

44. `deleteRear()` and FIFO

A normal queue follows FIFO.

That means:

```
The first value inserted must be the first value removed.
```

But `deleteRear()` removes the newest value.

Example:

```
Deque = 5 20 30
```

The value `30` entered after `5` and `20`.

If we call:

```
deleteRear();
```

Then `30` leaves first.

So a deque can break strict FIFO behavior.

Remember:

```
A deque is more flexible than a normal queue.
```

---

45. Time Complexity of `deleteRear()`

`deleteRear()` is:

```
O(1)
```

Why?

Because it only:

- checks if the deque is empty
- reads the rear value
- moves `rear` backward
- returns one value

It does not use a loop.

Part E: `display()`

---

46. What Is `display()`?

`display()` prints all valid values in the deque.

In this simple array design, valid values are from:

```
front + 1
```

to:

```
rear
```

---

47. `display()` Code

```
void display(Deque dq) {
    for (int i = dq.front + 1; i <= dq.rear; i++) {
        cout << dq.Q[i] << " ";
    }
    cout << endl;
}
```

---

48. `display()` Logic Step by Step

To display the deque:

1. Start from `front + 1`.
2. Print each value.
3. Continue until `rear`.
4. Stop after printing the rear value.

Simple meaning:

```
Print only the logical deque values.
Ignore values before or after the valid range.
```

---

49. Why Does `display()` Start at `front + 1`?

Because `front` points before the first valid element.

So the first valid element is:

```
front + 1
```

Example:

```
front = 0
rear = 2
Array = [10] [20] [30]
```

The valid range is:

```
front + 1 = 1
to
rear = 2
```

So `display()` prints:

```
20 30
```

It does not print `10` because `front` has moved past it.

50. Time Complexity of `display()`

`display()` is:

```
O(n)
```

Why?

Because it prints every valid element.

If the deque has 3 valid elements, it prints 3 values.

If the deque has 100 valid elements, it prints 100 values.

So the time depends on the number of values.

Part F: Complete Phase 8 Program

---

51. Complete C++ Program

```cpp
#include <iostream>
using namespace std;

struct Deque {
    int size;
    int front; // points before first valid element
    int rear;  // points at last valid element
    int *Q;
};

void create(Deque *dq, int size) {
    dq->size = size;
    dq->front = dq->rear = -1;
    dq->Q = new int[size];
}

void insertRear(Deque *dq, int x) {
    if (dq->rear == dq->size - 1) {
        cout << "No space at rear\n";
    } else {
```

```
        dq->rear++;
        dq->Q[dq->rear] = x;
    }
}

int deleteFront(Deque *dq) {
    if (dq->front == dq->rear) {
        cout << "Deque empty\n";
        return -1;
    }

    dq->front++;
    return dq->Q[dq->front];
}

int deleteRear(Deque *dq) {
    if (dq->front == dq->rear) {
        cout << "Deque empty\n";
        return -1;
    }

    int x = dq->Q[dq->rear];
    dq->rear--;
    return x;
}

void insertFront(Deque *dq, int x) {
    if (dq->front == -1) {
        cout << "No space at front\n";
    } else {
        dq->Q[dq->front] = x;
        dq->front--;
    }
}

void display(Deque dq) {
    for (int i = dq.front + 1; i <= dq.rear; i++) {
        cout << dq.Q[i] << " ";
    }
    cout << endl;
}

int main() {
    Deque dq;
    create(&dq, 5);

    insertRear(&dq, 10);
    insertRear(&dq, 20);
```

```
    insertRear(&dq, 30);
    display(dq); // 10 20 30

    cout << "Delete front: " << deleteFront(&dq) << endl; // 10

    insertFront(&dq, 5);
    display(dq); // 5 20 30

    cout << "Delete rear: " << deleteRear(&dq) << endl; // 30
    display(dq); // 5 20

    delete[] dq.Q;
    return 0;
}
```

---

52. Complete Program Explanation

Header File

```
#include <iostream>
```

This allows us to use `cout`.

Namespace

```
using namespace std;
```

This allows us to write `cout` instead of `std::cout`.

Deque Structure

```
struct Deque {
    int size;
    int front;
    int rear;
    int *Q;
};
```

This stores all deque information in one structure.

Create Deque

```
Deque dq;
create(&dq, 5);
```

This creates a deque with array size `5`.

After this:

```
size = 5
front = -1
rear = -1
```

The deque is empty.

Insert Values at Rear

```
insertRear(&dq, 10);
insertRear(&dq, 20);
insertRear(&dq, 30);
```

The deque becomes:

```
10 20 30
```

Display

```
display(dq);
```

This prints:

```
10 20 30
```

Delete from Front

```
deleteFront(&dq);
```

This removes:

```
10
```

Now the deque is:

```
20 30
```

Insert at Front

```
insertFront(&dq, 5);
```

This inserts `5` at the front.

Now the deque is:

```
5 20 30
```

Delete from Rear

```
deleteRear(&dq);
```

This removes:

```
30
```

Now the deque is:

```
5 20
```

Free Memory

```
delete[] dq.Q;
```

The array was created using `new[]`, so it should be released using `delete[]`.

This prevents memory leaks.

---

53. Expected Output

```
10 20 30
Delete front: 10
5 20 30
Delete rear: 30
5 20
```

This output shows that:

- values can be inserted at the rear
- values can be deleted from the front
- values can be inserted at the front
- values can be deleted from the rear

So the program proves the main deque idea.

Part G: Full Dry Run

---

54. Starting State

After:

```
create(&dq, 5);
```

State:

```
size = 5
front = -1
rear = -1
Deque = empty
```

---

55. Step 1: `insertRear(10)`

Operation:

```
insertRear(&dq, 10);
```

What happens:

```
rear moves from -1 to 0
Q[0] = 10
```

State:

```
front = -1
rear = 0
Deque = 10
```

---

56. Step 2: `insertRear(20)`

Operation:

```
insertRear(&dq, 20);
```

What happens:

```
rear moves from 0 to 1
Q[1] = 20
```

State:

```
front = -1
rear = 1
Deque = 10 20
```

---

57. Step 3: `insertRear(30)`

Operation:

```
insertRear(&dq, 30);
```

What happens:

```
rear moves from 1 to 2
Q[2] = 30
```

State:

```
front = -1
rear = 2
Deque = 10 20 30
```

---

58. Step 4: `deleteFront()`

Operation:

```
deleteFront(&dq);
```

What happens:

```
front moves from -1 to 0
Q[0] = 10 is returned
```

Deleted value:

```
10
```

State:

```
front = 0
rear = 2
Deque = 20 30
```

---

59. Step 5: `insertFront(5)`

Operation:

```
insertFront(&dq, 5);
```

What happens:

```
Q[0] = 5
front moves from 0 to -1
```

State:

```
front = -1
rear = 2
Deque = 5 20 30
```

---

60. Step 6: `deleteRear()`

Operation:

```
deleteRear(&dq);
```

What happens:

```
Q[2] = 30 is returned
rear moves from 2 to 1
```

Deleted value:

```
30
```

Final state:

```
front = -1
rear = 1
Deque = 5 20
```

---

61. Dry Run Table

Step	Operation	`front`	`rear`	Logical Deque
1	Create deque	-1	-1	Empty
2	`insertRear(10)`	-1	0	10
3	`insertRear(20)`	-1	1	10, 20
4	`insertRear(30)`	-1	2	10, 20, 30
5	`deleteFront()`	0	2	20, 30
6	`insertFront(5)`	-1	2	5, 20, 30
7	`deleteRear()`	-1	1	5, 20

Part H: Restricted Deques

---

62. What Is a Restricted Deque?

A restricted deque is a deque where one type of operation is limited.

A normal deque allows:

```
insert at front
insert at rear
delete from front
delete from rear
```

A restricted deque does not allow all four operations freely.

There are two common restricted deque types:

1. Input-restricted deque
2. Output-restricted deque

63. Input-Restricted Deque

An **input-restricted deque** restricts insertion.

That means insertion is allowed from only one end.

Usually:

```
Insertion: rear only
Deletion: front and rear
```

So an input-restricted deque allows:

```
insertRear()
deleteFront()
deleteRear()
```

But it does not allow:

```
insertFront()
```

Simple meaning:

```
You can insert from only one side, but you can delete from both sides.
```

---

64. Input-Restricted Deque Example

Suppose we insert values:

```
insertRear(10)
insertRear(20)
insertRear(30)
```

Deque:

```
10 20 30
```

Now we can delete from the front:

```
deleteFront() -> removes 10
```

Or delete from the rear:

```
deleteRear() -> removes 30
```

But we cannot insert at the front.

So this is not allowed:

```
insertFront(5)
```

---

65. Output-Restricted Deque

An **output-restricted deque** restricts deletion.

That means deletion is allowed from only one end.

Usually:

```
Insertion: front and rear
Deletion: front only
```

So an output-restricted deque allows:

```
insertFront()
insertRear()
deleteFront()
```

But it does not allow:

```
deleteRear()
```

Simple meaning:

```
You can insert from both sides, but you can delete from only one side.
```

---

66. Output-Restricted Deque Example

Suppose the deque is:

```
10 20
```

We can insert at the rear:

```
insertRear(30)
```

Deque becomes:

```
10 20 30
```

We can also insert at the front:

```
insertFront(5)
```

Deque becomes:

```
5 10 20 30
```

But deletion is only allowed from the front.

So this is allowed:

```
deleteFront()
```

But this is not allowed:

```
deleteRear()
```

---

67. Restricted Deque Summary

Type	Insertion Allowed	Deletion Allowed
Input-restricted deque	Rear only	Front and rear
Output-restricted deque	Front and rear	Front only

Easy memory trick:

```
Input-restricted = insertion is restricted.
Output-restricted = deletion/output is restricted.
```

Part I: Complexity Summary

---

68. Time Complexity Table

Operation	Time Complexity	Why?
`insertRear()`	O(1)	Moves `rear` and stores one value
`deleteFront()`	O(1)	Moves `front` and returns one value
`insertFront()`	O(1)	Stores one value and moves `front` backward
`deleteRear()`	O(1)	Reads rear value and moves `rear` backward
`display()`	O(n)	Prints all valid values

---

69. Why the Four Main Operations Are O(1)

The four main deque operations are fast because each one changes only one end.

For example:

```
insertRear() changes rear
deleteFront() changes front
insertFront() changes front
deleteRear() changes rear
```

None of these operations shifts all elements.

So each operation takes constant time:

```
O(1)
```

---

70. Space Complexity

The array deque uses an array of fixed size.

So the space complexity is:

```
O(size)
```

If the deque has an array of size 5, it uses space for 5 integers.

If the deque has an array of size 100, it uses space for 100 integers.

Part J: Common Beginner Mistakes

---

71. Mistake 1: Thinking Deque and Dequeue Are the Same

These two words look similar, but they are different.

Word	Meaning
`deque` / `DEQueue`	A double ended queue data structure
`dequeue()`	A delete operation from a normal queue

Remember:

```
Deque = data structure.
Dequeue = operation.
```

---

72. Mistake 2: Thinking a Deque Always Follows FIFO

A normal queue follows FIFO.

But a deque can delete from the rear.

Example:

```
Deque = 10 20 30
```

If we call:

```
deleteRear()
```

Then `30` leaves first.

So a deque does not always behave like a strict FIFO queue.

---

73. Mistake 3: Forgetting That `front` Points Before the First Valid Element

In this design:

```
front does not directly point to the first valid value.
```

The first valid value is at:

```
front + 1
```

So if:

```
front = 0
rear = 2
```

The valid values are from:

```
1 to 2
```

Not from `0` to `2`.

---

74. Mistake 4: Starting `display()` at `front` Instead of `front + 1`

Wrong idea:

```
for (int i = front; i <= rear; i++)
```

Correct idea:

```
for (int i = front + 1; i <= rear; i++)
```

Why?

Because `front` points before the first valid element.

---

75. Mistake 5: Forgetting That `insertFront()` Needs Front Space

In this simple linear array implementation, `insertFront()` needs space before the first valid element.

If:

```
front == -1
```

there is no front space.

So `insertFront()` cannot insert.

It becomes possible after `deleteFront()` moves `front` forward.

---

76. Mistake 6: Forgetting to Move `rear` Backward in `deleteRear()`

When deleting from the rear, we must do:

```
dq->rear--;
```

If we forget this, the deleted value will still be treated as part of the deque.

---

77. Mistake 7: Forgetting to Free Dynamic Memory

The array is created using:

```
dq->Q = new int[size];
```

So it should be released using:

```
delete[] dq.Q;
```

If we forget this, memory is wasted.

Part K: Important Rules to Remember

---

78. Main Deque Rules

A deque allows four main operations:

```
insertRear()
deleteFront()
insertFront()
deleteRear()
```

A normal queue only allows:

```
insert at rear
delete from front
```

A deque allows:

```
insert at both ends
delete from both ends
```

---

79. Important Conditions

Condition	Meaning
`front == rear`	Deque is empty
`rear == size - 1`	No space at rear
`front == -1`	No space at front in this simple version
`front + 1`	First valid element
`rear`	Last valid element

---

80. Important Operation Meanings

Operation	Simple Meaning
`insertRear(x)`	Add `x` after the current rear value
`deleteFront()`	Remove the front value
`insertFront(x)`	Add `x` before the current first valid value
`deleteRear()`	Remove the rear value
`display()`	Print all valid values from front to rear

---

81. Deque Compared With Previous Queue Types

Queue Type	Main Idea	Main Limitation or Feature
Linear array queue	Uses array with `front` and `rear`	Rear only moves forward
Circular queue	Uses modulo to reuse array spaces	Fixed array size
Linked-list queue	Uses dynamic nodes	Grows as memory allows
Deque	Allows operations at both ends	More flexible than normal queue

---

82. Real-Life Analogy

Imagine a line where people can enter and leave from both doors.

There is a front door and a back door.

In a normal queue:

```
People enter from the back and leave from the front.
```

In a deque:

```
People can enter from the front or back.
People can leave from the front or back.
```

That is why it is called a double ended queue.

---

83. Final Summary

A **double ended queue**, or **deque**, is a queue where both ends are active.

A normal queue is strict:

```
Insert at rear.
Delete from front.
```

A deque is flexible:

```
Insert at rear.
Delete from front.
Insert at front.
Delete from rear.
```

The four main operations are:

```
insertRear()
deleteFront()
```

```
insertFront()
deleteRear()
```

In the simple array implementation:

```
front points before the first valid element.
rear points at the last valid element.
```

The empty condition is:

```
front == rear
```

The first valid element is:

```
front + 1
```

The last valid element is:

```
rear
```

The four main deque operations are all:

```
O(1)
```

because they only move `front` or `rear`.

`display()` is:

```
O(n)
```

because it prints all valid elements.

The two restricted deque types are:

```
Input-restricted deque: insertion is restricted.
Output-restricted deque: deletion is restricted.
```

Final takeaway:

```
Deque = a queue where insertion and deletion can happen at both ends.
```